MEDINOVA

Producto Integrador de Aprendizaje (PIA)

Componentes de Lógica de Negocios · Proyecto final

Java 21 · Spring Boot · PostgreSQL · Docker Compose · OpenAPI · GitHub Actions

Artefactos de referencia: Unidades 2 y 3 · 21 de octubre de 2026

Contenido

1. Resumen ejecutivo

2. Problema, objetivos y alcance

3. Análisis del negocio

4. Arquitectura DDD y UML

5. Patrones y SOLID

6. Reglas esenciales y tardanza

7. API y persistencia

8. Calidad y automatización

9. Contenedores, configuración y despliegue

10. Trazabilidad

11. Riesgos y pendientes

12. Plan de demostración y defensa

13. Conclusión

1. Resumen ejecutivo

MediNova es una clínica privada de tres sucursales con cerca de 35 médicos y horarios variables. El PIA integra el trabajo previo de análisis con una implementación demostrativa en Java 21, Spring Boot y PostgreSQL. El núcleo ejecutable modela el ciclo de cita, validación de agenda, llegada, tardanza, decisión médica explícita, cancelación y no-show. Se incorporan Specifications combinables, máquina de estados, eventos de dominio, API REST/OpenAPI, pruebas automatizadas, Docker Compose y workflow de CI.

2. Problema, objetivos y alcance

La operación AS-IS depende de una agenda compartida y acciones manuales. Esto favorece citas duplicadas, baja visibilidad entre sucursales, cancelaciones sin evidencia uniforme y dificultad para medir no-shows e historial. Los objetivos son reducir duplicidades, mejorar disponibilidad, registrar eventos y excepciones, preservar continuidad de consulta y habilitar indicadores. El alcance y los requerimientos refinados están en docs/BRD.md.

3. Análisis del negocio

Se conserva como base el BRD, historias de usuario, BPMN y glosario aportados en el ZIP. El paquete incluye 20 historias HU-01 a HU-20 con escenarios Gherkin; supera el mínimo de 15. Los actores son paciente, recepcionista, médico, administración y aseguradora. Las reglas pendientes de validación (permisos formales, sobrecupo, reincidencia de no-show, ventanas de cancelación) no se convierten en políticas institucionales inventadas.

4. Arquitectura DDD y UML

Appointment representa el agregado raíz del ciclo de vida de una cita; encapsula invariantes y transiciones. Los identificadores de paciente, médico y sucursal se mantienen como referencias. AppointmentService coordina casos de uso; AppointmentRepository encapsula persistencia; API Controller expone contrato REST. Los diagramas de dominio y secuencia se incluyen en docs/architecture/diagrams y se conservan los diagramas originales en originales/.

5. Patrones y SOLID

Specification concentra validaciones composables (paciente activo, médico activo, sucursal activa, autorización y horario válido) con AND/OR/NOT. State Machine restringe cambios de estado. Domain Events representan confirmación, llegada, decisión y cancelación. Repository separa persistencia del dominio; Service coordina el caso de uso. La separación API/aplicación/dominio/infraestructura apoya SRP y DIP; las interfaces de repositorio desacoplan servicios del mecanismo de almacenamiento.

6. Reglas esenciales y tardanza

La hora de llegada se compara con el inicio programado y se calculan minutos de tardanza. El sistema no resuelve automáticamente si el paciente debe ser atendido. La persona médica asignada registra ATENDER, ESPERAR o REPROGRAMAR. Una tardanza de 15 minutos o menos activa la consulta al médico según el caso original; retrasos mayores normalmente conducen a ofrecer reagenda, sin sustituir la decisión clínica por una regla automática.

7. API y persistencia

La API expone GET/POST /api/appointments y acciones para confirmar, registrar llegada, registrar decisión, cancelar y marcar no-show. Springdoc genera Swagger UI en /swagger-ui.html. PostgreSQL es la base de datos de Compose; H2 se utiliza en pruebas de integración. Los códigos de respuesta previstos incluyen 201 al crear, 400 por validación, 403 por médico no asignado, 404 por recurso ausente y 409 por transición o conflicto.

Método/ruta	Uso	Respuesta
GET /api/appointments	Listar citas	200
POST /api/appointments	Crear cita	201/400/409
GET /api/appointments/{id}	Consultar	200/404
POST /{id}/arrival	Registrar llegada	200/409
POST /{id}/decision	Decisión médica	200/403/409
POST /{id}/cancel	Cancelar	200/409
POST /{id}/no-show	No-show	200/409

8. Calidad y automatización

Hay pruebas unitarias de invariantes de dominio y pruebas de integración HTTP con perfil H2. JaCoCo está configurado con umbral de 80% de instrucciones en `mvn verify`; la cobertura real debe confirmarse ejecutando el build, no se afirma sin reporte. GitHub Actions ejecuta Maven verify en push y pull request. La ejecución del workflow queda pendiente de publicar el proyecto en un repositorio accesible.

9. Contenedores, configuración y despliegue

Docker Compose define PostgreSQL, aplicación y un perfil de ejecución de tests. `.env.example` separa configuración; no contiene secretos reales. Para despliegue se recomienda Railway, Render o plataforma equivalente con PostgreSQL administrado, variables DB_URL/DB_USER/DB_PASSWORD/PORT y Swagger activo. No se inventa una URL pública: requiere cuenta y credenciales del equipo.

10. Trazabilidad

La matriz docs/Traceability.md conecta reglas BR-01 a BR-09 con historias, clases, endpoints y pruebas. Señala explícitamente brechas: resultado clínico, historial e indicadores no están completos en esta versión del código; tampoco hay pruebas de integración de colisiones concurrentes. La matriz debe actualizarse tras completar esos módulos.

Regla	Historia	Componente	Estado
BR-01	HU-04	Appointment/Specification	Implementado
BR-03	HU-05	Service/Repository	Parcial
BR-05/06	HU-07/08	State Machine	Implementado
BR-08	HU-10	Domain Event	En memoria
FR-11	HU-13	Resultado clínico	Pendiente
FR-12	HU-14	Historial	Pendiente
FR-13	HU-15	Indicadores	Pendiente

11. Riesgos y pendientes

El prototipo no debe exponerse con datos reales. Falta autenticación/autorización robusta; el identificador del médico se envía en la petición y no deriva de una identidad autenticada. La comprobación de solapamientos no basta para resolver carreras concurrentes sin restricciones/aislamiento adicional. También falta migración de esquema versionada, auditoría persistente inmutable, UI operativa, módulos completos de pacientes/médicos/sucursales y panel de indicadores.

12. Plan de demostración y defensa

Demostrar Swagger; crear cita con intervalo válido; confirmar; registrar llegada con retraso; observar minutos calculados y ausencia de decisión automática; registrar decisión como médico asignado; probar decisión de otro médico y transición inválida; cancelar otra cita y demostrar evento; ejecutar `mvn verify`; iniciar Compose. Explicar cómo las Specifications aplican reglas, cómo la máquina de estados evita cambios inválidos y cómo la matriz conecta requerimientos con pruebas.

13. Conclusión

El PIA convierte los artefactos de MediNova en una base ejecutable y trazable, con dominio explícito, contrato REST, persistencia, automatización y despliegue reproducible. La implementación es una entrega académica incremental y honesta: el núcleo de citas está modelado, pero los módulos clínicos, indicadores, seguridad productiva, despliegue real y evidencia de cobertura deben completarse y verificarse antes de afirmar cumplimiento total del PIA.

Anexo: evidencia pendiente

Antes de entregar como versión final, adjuntar ejecución real de Maven/JaCoCo, workflow CI verde, URL de GitHub, URL del despliegue y capturas de la defensa. No se afirma que estos elementos externos estén activos desde este entorno.